Agent Input/Output Contracts

Marketing Agent POC

Each agent has a defined input scope and output schema. Agents receive only the context they need -- not the full pipeline state.

---

vision.py -- Product Analyst

Input: - `image_path: str` -- path to saved product image file - `description: str` -- user-supplied short product description

Output:

```
{
  "product_type": "water bottle",
  "product_tags": ["insulated", "eco-friendly", "BPA-free", "bamboo"],
  "mood": "energetic",
  "target_signals": ["fitness enthusiast", "eco-conscious", "outdoor lifestyle"],
  "suggested_hashtags": ["#EcoHydration", "#LiveGreen"],
  "model_used": "nvidia/nemotron-nano-12b-v2-vl:free"
}
```

What it does NOT receive: color data, audience data, tone preference What it does NOT produce: persona, copy, compliance flags

---

color.py -- Color Extractor

Input: - `image_path: str`

Output:

```
{
  "dominant": "#4caf50",
  "palette": ["#4caf50", "#2e7d32", "#81c784"]
}
```

Note: No LLM call. PIL only. Always succeeds or returns a neutral fallback.

---

audience.py -- Account Planner

Input: - `description: str` - `vision_data: dict` -- full vision output (product_type, product_tags, mood, target_signals) - `doc_context: list[str]` -- optional RAG chunks

Output:

```
{
  "persona_label": "Eco-Conscious Millennial Athlete",
  "age_range": "25-34",
  "interests": ["sustainability", "fitness", "zero-waste lifestyle"],
  "platform_behavior": "follows sustainability influencers, engages with eco content"
}
```

What it does NOT receive: color data, previous copy, CTA scores Constraint: persona_label must be specific -- never "general audience" or "everyone"

---

copywriter.py -- Creative

Input: - `description: str` - `vision_data: dict` - `color_data: dict` - `tone: str` - `audience_data: dict` -- full audience output - `doc_context: list[str]` -- optional RAG chunks

Output:

```
{
  "variant_1": "Stay hydrated, stay green. 24h cold, zero plastic. #LiveGreen",
  "variant_2": "Insulated. BPA-free. 24h cold. Your gym companion. Shop now."
}
```

Enforced angles: variant_1 opens with emotional hook; variant_2 opens with product benefit
Enforced limit: each variant ?150 characters

---

guardrails.py -- Compliance Reviewer

Input: - `copy: dict` -- {variant_1: str, variant_2: str}

Output:

```
{
  "clean_copy": {"variant_1": "...", "variant_2": "..."},
  "flags": [
    {
      "variant": "variant_1",
      "matched": "guaranteed",
      "reason": "Guarantee claim -- requires substantiation"
    }
  ]
}
```

What it does NOT receive: persona, vision data, tone What it does NOT do: modify copy text -- clean_copy is always identical to input What it uses: pure Python regex, zero LLM calls

---

cta_optimizer.py -- Media Strategist

Input: - `copy: dict` -- clean copy from guardrails output - `tone: str` - `audience_data: dict` - `doc_context: list[str]` -- optional

Output:

```
{
  "variant_1": {
    "original_cta": "Shop now",
    "score": 7,
    "suggestion": "Shop now -- limited stock",
    "reasoning": "adds urgency without hype"
  },
  "variant_2": {
    "original_cta": "Get yours",
    "score": 6,
    "suggestion": "Get yours today",
    "reasoning": "time anchoring improves intent"
  }
}
```

---

rag.py -- Knowledge Retrieval

Input (retrieve): - `query: str` -- description + product tags joined

Output: - `list[str]` -- up to RAG_TOP_K text chunks from ChromaDB

Input (ingest_files): - `paths: list[str]` -- PDF, TXT, or MD file paths

Output: - `{"ingested": int, "skipped": int}`

---

assistant.py -- Router and Analyst

Functions: - `suggest(output)` -> `str` -- proactive suggestion bullets on page load - `get_routing(output, user_message, tone, age, formality, history)` -> `dict` - Returns: `{agents_to_run: list, reasoning: str, routing_label: str}` - `map_sliders_to_params(tone, age, formality)` -> `dict` - Returns: `{tone: str, age_hint: str, formality_instruction: str}`